PRECISION UNIFIED AUTOMATION FRAMEWORK

Project Documentation

Submitted To	**Veeva Systems**
Application	https://automationexercise.com
Version	1.0.0 | April 2026

TEAM QALASERS

Team Member	Email	Phone
Bandi Poorna Sri Sailaja	sailubandi33@gmail.com	6300655517
Andra Tulasi Lakshmi Tanya	tanyakonapala@gmail.com	6302601497
Nerella N S Venkata Siva Sai Rihitha	rihitha9@gmail.com	9290069793
Rambha Rasmitha Lekha	rambhasrasmi7024@gmail.com	91214 87799

GitHub: https://github.com/sailubandi/Precision-Unified-Automation-Framework

1. Project Overview

This framework is designed to automate both the User Interface (UI) and API layers of the AutomationExercise web application. It follows industry-standard practices to ensure reliable, maintainable, and scalable test automation coverage across critical user journeys such as user registration, login, and cart management.

The framework adopts a Behaviour Driven Development (BDD) approach using Cucumber, enabling test scenarios to be written in plain English (Gherkin syntax) that are understandable by both technical and non-technical stakeholders. It is structured as a Maven multi-module project, separating UI and API concerns into dedicated modules while sharing common utilities across both.

1.1 Project Goals

- Design a unified Hybrid Automation framework covering both UI and REST API testing.
- Implement a clean Maven multi-module architecture with strict separation of concerns.
- Apply the Page Object Model (POM) pattern to isolate UI locators from test logic.
- Implement a layered API design: Client → Request Builder → Validator.
- Support parallel test execution to reduce overall test cycle time.
- Provide structured logging (Log4j2) and reporting (Allure + Cucumber HTML + Extent Reports).
- Ensure zero hard-coding: all configuration externalized to properties files.
- Capture automatic screenshots on UI test failure for faster defect detection.

1.2 Scope of Testing

The framework covers two layers of testing — UI and API — targeting the most critical functional areas of the AutomationExercise application.

UI Test Scope

Test Case	Description
TC1 — User Registration	Registers a new user with full account details, verifies account creation, and cleans up by deleting the account.
TC2 — Valid User Login	Logs in with valid credentials and verifies the user is successfully authenticated.
TC3 — Invalid User Login	Attempts login with invalid credentials and verifies the appropriate error message is displayed.
TC4 — Add Product to Cart	Navigates to the products page, adds two products to the cart, and validates product names, quantities, and total prices.
TC5 — Remove Product from Cart	Adds a product to the cart, navigates to the cart page, removes the product, and verifies it no longer appears.

API Test Scope

Test Case	Description
TC1 — Get All Products	Sends a GET request to the products list endpoint, validates status code 200, response body, and JSON schema.

TC2 — Search Product	Sends a POST request to search for products by name (top, tshirt, jean). Validates status code and schema compliance.
TC3 — Create User Account	Sends a POST request to register a new user account. Validates response code and success/duplicate message.
TC4 — Delete User Account	Sends a DELETE request to delete an existing user account. Validates the response code.
TC5 — Update User Account	Sends a PUT request to update an existing user account. Validates response code and confirms data update.
TC6 — Negative Validation	Tests invalid email format, missing required fields, and non-existent product searches to verify proper error handling.

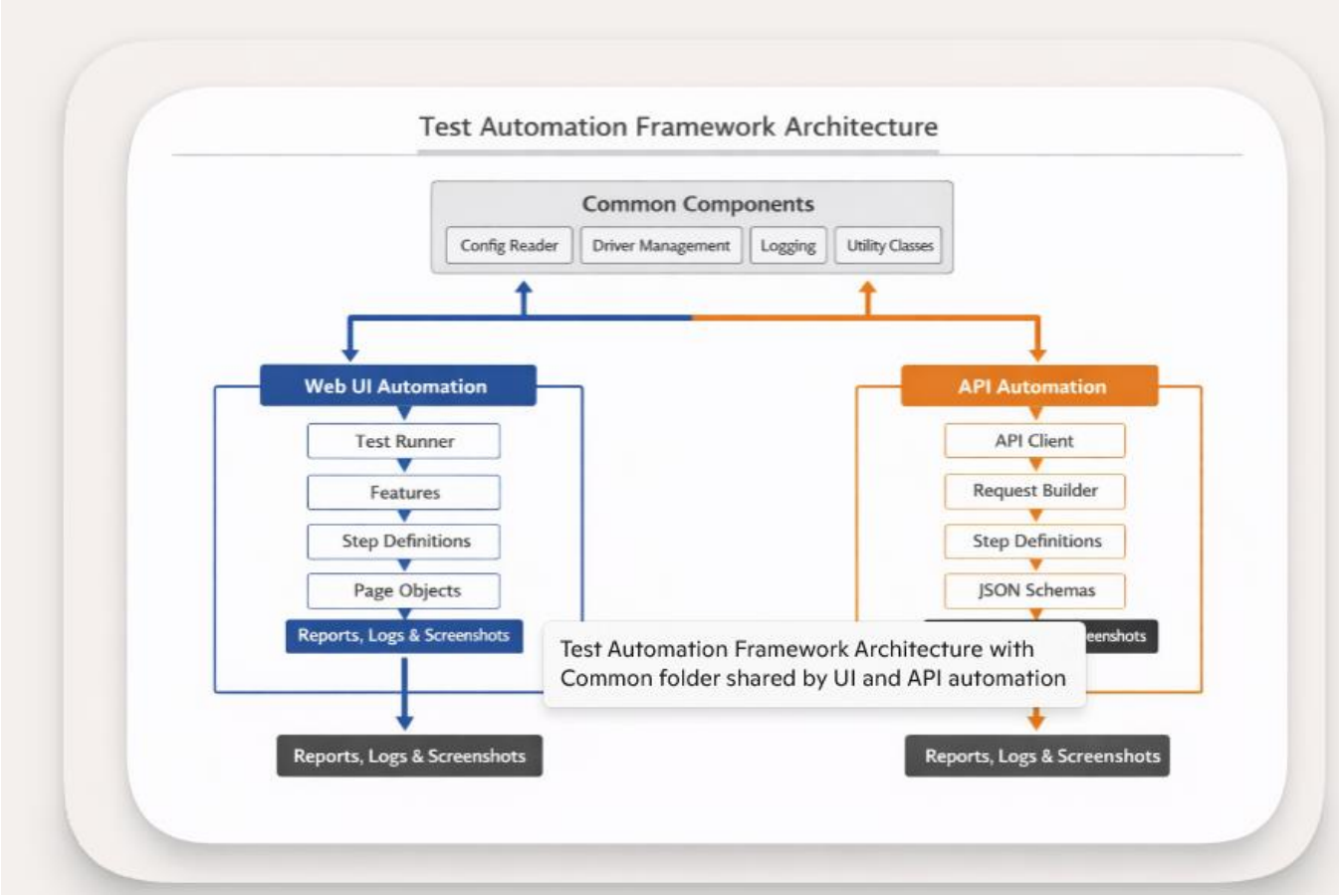
Out of Scope

- Order placement, payment, and checkout flows
- Subscription functionality
- Non-functional testing such as performance, security, or accessibility testing

2. Architecture

The framework is structured as a Maven multi-module project with three primary modules and a parent POM that orchestrates the entire build. Each module has a single, clearly defined responsibility.

2.1 Framework Architecture Diagram



2.2 Module Overview

Module	Responsibility
Parent POM	Manages all child modules, centralizes dependency versions via <dependencyManagement>, and defines shared build properties.
Common	Provides shared utilities (DriverFactory, ConfigReader, WaitUtils, TestDataUtils) consumed by both UI and API modules. Compiled as a regular JAR.
UIAutomation	Contains all Selenium WebDriver automation using Page Object Model, Cucumber BDD, hooks, and Extent Report generation.
APIAutomation	Contains all RestAssured-based API tests, request builders, validators, JSON schema validation, and Allure reporting.

2.3 Framework Directory Structure

```
HybridAutomation/
|-- APIAutomation/
|   |-- src/test/java/
|   |   |-- client/              # ApiClient.java — centralized HTTP client
|   |   |-- com/framework/api/
|   |   |   |-- builder/         # RequestBuilder.java
|   |   |   |-- factory/         # UserDataFactory.java, ProductDataFactory.java
|   |   |   |-- pojo/            # User.java (data model)
|   |   |   |-- runner/          # ApiTestRunner.java
|   |   |   |-- stepdefinitions/ # TC1-TC6 step definitions + CommonSteps
|   |   |   `-- validator/       # ResponseValidator.java
|   |-- src/test/resources/
|   |   |-- features/            # TC1-TC6 .feature files
|   |   |-- schemas/             # JSON schema validation files
|   |   `-- config.properties
|   `-- testng-api.xml
|-- UIAutomation/
|   |-- src/test/java/
|   |   |-- PageObjects/         # HomePage, LoginPage, RegisterPage, CartPage, ProductPage
|   |   |-- hooks/               # Hooks.java (@Before/@After)
|   |   |-- runner/              # TestRunner.java (parallel DataProvider)
|   |   |-- reports/             # ExtentManager.java, ExtentTestManager.java
|   |   |-- stepdefinitions/     # TC1-TC5 step definitions
|   |   `-- utils/               # ExcelUtils.java
|   |-- src/test/resources/
|   |   |-- Feature/             # Register, Login, Cart .feature files
|   |   `-- config.properties
|   `-- testng-ui.xml
|-- Common/
|   |-- src/main/java/
|   |   |-- config/              # ConfigReader.java
|   |   |-- driver/              # DriverFactory.java, DriverManager.java
|   |   |-- logger/              # LogManager.java
|   |   `-- utils/               # WaitUtils.java, TestDataUtils.java, AdBlockerUtil.java
|   `-- pom.xml
|-- pom.xml                      # Parent Maven configuration
|-- testng.xml                   # Unified test execution
`-- README.md
```

2.4 Architecture Component Roles

- **Manages dependencies, plugins, and build configuration for all modules. Centralizes version properties for Selenium, Cucumber, TestNG, RestAssured, Allure, and Log4j2.**Parent Module (pom.xml):
- **Provides shared utilities consumed by both UI and API modules — DriverFactory, ConfigReader, WaitUtils, TestDataUtils, and AdBlockerUtil. Compiled as a JAR and referenced as a dependency.**Common Module:
- **Handles frontend web application testing using Selenium WebDriver, Page Object Model pattern, and Cucumber BDD. Produces Extent Reports and captures screenshots on failure.**UIAutomation Module:
- **Performs backend REST API testing using RestAssured. Implements layered design: ApiClient → RequestBuilder → ResponseValidator. Produces Allure and Cucumber HTML reports.**APIAutomation Module:
- **Control test execution flow for both UI and API modules. The UI runner supports parallel scenario execution via @DataProvider(parallel = true).**TestNG Runners:

3. Module Breakdown

3.1 Common Module

Purpose: Provides shared utilities consumed by both the UIAutomation and APIAutomation modules, avoiding code duplication. This module is compiled as a regular JAR and referenced as a Maven dependency by both test modules.

Class	Description
ConfigReader	Loads config.properties at class initialization using a static block. Provides a static get(String key) method that throws a RuntimeException if a key is not found, ensuring misconfiguration is caught early.
DriverFactory	Manages WebDriver lifecycle. Handles driver initialization for Chrome, Edge, and Firefox via WebDriverManager. Applies implicit waits and delegates storage to DriverManager.
DriverManager	Stores the WebDriver in a ThreadLocal<WebDriver> instance to support parallel execution. Provides getDriver(), setDriver(), and unload() methods ensuring thread isolation.
WaitUtils	Provides explicit wait strategies using WebDriverWait. Includes methods for waiting for element visibility, clickability, and presence.
TestDataUtils	Provides test data helper methods used by both API and UI test modules, including random string and email generation.
AdBlockerUtil	Builds ChromeOptions configured to block ad-serving domains and suppress automation flags, preventing ads from interfering with UI tests.
LogManager	Wraps Log4j2's LogManager. Exposes a static getLogger() method used throughout both modules to obtain named loggers for consistent, structured logging.

3.2 UIAutomation Module

Purpose: Contains all components required to automate and execute UI test scenarios against the AutomationExercise web application using Selenium WebDriver and Cucumber.

Page Objects

Class	Description
HomePage	Locators and actions for the home page — clicking Signup/Login, Products, Cart buttons, and verifying the logged-in username.
LoginPage	Handles both the login form and signup name/email form on the same page. Provides methods to enter credentials, click buttons, and retrieve displayed messages.
RegisterPage	Fills in the full account registration form including title, password, date of birth, address fields, and country dropdown using Selenium's Select class. Uses JavaScript executor to handle the Create Account button click reliably.
AccountPage	Retrieves account creation and deletion confirmation messages, and provides Continue button actions for post-registration/deletion flows.

ProductPage	Manages product interactions on the products listing page — adding the first and second products to cart, clicking Continue Shopping, navigating to the cart, storing product names, and calculating total price validation.
CartPage	Handles cart page interactions — verifying the cart table is displayed, retrieving product names, clicking the delete button, waiting for product removal, and checking product visibility.

Hooks

Class	Description
Hooks	@Before hook initializes WebDriver via DriverFactory and creates an ExtentReports test entry. @After hook captures a screenshot on failure (saved to Screenshots/ and embedded in both Cucumber and Extent reports), passes the scenario if successful, then quits the driver and flushes the report.

Runner

Class	Description
TestRunner	Extends AbstractTestNGCucumberTests and overrides the scenarios() DataProvider with @DataProvider(parallel = true). Features loaded from src/test/resources/Feature. Glue set to stepdefinitions and hooks packages. Produces Cucumber HTML report to reports/cucumber.html.

Reports

Class	Description
ExtentManager	Singleton managing the ExtentReports instance. Initializes report with HTML configuration, sets document title and report name, and provides flush() to write the report to disk.
ExtentTestManager	Thread-safe manager for ExtentTest instances using ThreadLocal. Provides setTest(), getTest(), and unload() methods so each parallel thread has its own test node in the report.

Step Definitions (UI)

Class	Description
TC1_RegisterSteps	Implements the full user registration flow: navigate to the site, enter signup details, fill registration form, verify account created, then delete the account.
TC2_LoginSteps	Implements valid login flow: navigate to login page, enter credentials, verify successful authentication and logged-in username.
TC3_InvalidLoginSteps	Implements negative login scenario: attempts login with invalid email/password and asserts the error message is displayed.
TC4_AddToCartSteps	Navigates to products page, adds two items to the cart, continues shopping, then validates both product names, quantities, and pricing in the cart.

TC5_RemoveFromCartSteps	Adds a product to the cart, navigates to the cart page, removes the product, and verifies the product is no longer displayed.

3.3 APIAutomation Module

Purpose: Performs backend REST API testing using RestAssured. Implements a layered design separating HTTP client concerns from request construction and response validation.

Client & Builder Layer

Class	Description
ApiClient	Centralized HTTP client built on RestAssured. Provides get(), post(), put(), delete() methods. Manages base URI configuration, sets Content-Type headers, and returns Response objects for validation.
RequestBuilder	Constructs HTTP requests with configurable headers, query parameters, form parameters, and JSON body payloads. Uses the Builder pattern for clean, readable request construction in step definitions.
ResponseValidator	Validates API responses including status code assertions, response body field checks, and JSON schema validation using RestAssured's JSON Schema Validator.

Factory & Data Layer

Class	Description
UserDataFactory	Generates complete User objects with all required fields populated from config.properties. Provides createValidUser() method used across TC3, TC4, and TC5 scenarios.
User (POJO)	Plain Old Java Object representing a user account with all fields matching the API contract: name, email, password, title, birth details, address fields.

Runner

Class	Description
ApiTestRunner	Extends AbstractTestNGCucumberTests. @CucumberOptions configured with features path, glue packages, pretty and html/json plugins, monochrome=true, and tag filter for @TC1 through @TC6. Produces cucumber.html and cucumber.json reports.

Step Definitions (API)

Class	Description
TC1_GetAllProductsSteps	Sends GET /api/productsList, asserts status 200, verifies the response contains a non-empty products array, and validates against the product-list JSON schema.

TC2_GetSingleProductSteps	Sends POST /api/searchProduct with search parameters (top, tshirt, jean). Validates status code, returned product details, and schema compliance for each search term.
TC3_CreateUserSteps	Sends POST /api/createAccount using UserDataFactory data. Validates response message (User created or Email already exists).
TC4_DeleteUserSteps	Sends DELETE /api/deleteAccount with user credentials. Validates the response code and deletion confirmation message.
TC5_UpdateUserSteps	Sends PUT /api/updateAccount with modified user data. Validates the response code and updated account confirmation.
TC6_NegativeValidationSteps	Tests malformed email validation, missing required field handling, and searches for non-existent products. Asserts appropriate error codes and messages are returned.
CommonSteps	Shared step definitions reused across multiple API test scenarios (e.g., common setup, base URL initialization).

4. Test Scenarios

4.1 UI Test Cases

TC1 — User Registration Flow

Feature File	Register.feature
Tag	@TC1 @Register @Smoke
Description	End-to-end registration: launch browser, navigate to site, click Signup/Login, enter name and email, fill registration form with full details, verify account created message, then delete the account.
Validation	Account creation confirmation message visible; login with new credentials succeeds; deletion confirmation message verified.

TC2 — Valid User Login

Feature File	Login.feature
Tag	@TC2 @Login @Smoke
Description	Login with valid credentials from config.properties and verify the logged-in username is displayed on the home page.
Validation	User dashboard visible; logged-in username displayed correctly.

TC3 — Invalid Login Validation

Feature File	Incorrectlogin.feature
Tag	@TC3 @Login @Negative
Description	Attempt login with an invalid email address and incorrect password. Verify the error message is displayed.
Validation	Error message "Your email or password is incorrect!" is visible on the login page.

TC4 — Add Products to Cart

Feature File	AddProductToCart.feature
Tag	@TC4 @Cart @E2E
Description	Navigate to the Products page, add two products to the cart using hover interactions and Continue Shopping, then navigate to the Cart page.
Validation	Both product names are present in the cart; quantities and total prices are correct.

TC5 — Remove Products from Cart

Feature File	RemoveFromCart.feature

Tag	@TC5 @Cart @Regression
Description	Add a product to the cart, navigate to the cart page, click the delete button for the product, and wait for it to be removed.
Validation	Product is no longer visible in the cart after deletion. Cart appears empty.

4.2 API Test Cases

TC1 — Get All Products

Endpoint	GET /api/productsList
Tag	@TC1 @Products @Smoke
Validation	Status code 200, response body contains non-empty products array, JSON schema matches product-list-schema.json.

TC2 — Search Product

Endpoint	POST /api/searchProduct
Tag	@TC2 @Search @Regression
Validation	Status code 200, returned products match the search term, schema compliance verified for each scenario. Search terms: "top", "tshirt", "jean".

TC3 — Create User Account

Endpoint	POST /api/createAccount
Tag	@TC3 @User @Critical
Validation	Status 201 (created) or 400 (email already exists). Response message validated. UserDataFactory provides all required fields.

TC4 — Delete User Account

Endpoint	DELETE /api/deleteAccount
Tag	@TC4 @User @Regression
Validation	Status 200 with account deleted message, or 404 if account not found. Credentials passed via request parameters.

TC5 — Update User Account

Endpoint	PUT /api/updateAccount
Tag	@TC5 @User @Regression
Validation	Status 200 with account updated message, or 404 if account not found. Confirms data update in response.

TC6 — Negative Validation Tests

Scenarios	Invalid email format, missing required fields, non-existent product search
Tag	@TC6 @Negative @Sanity
Validation	Appropriate HTTP error codes returned (400/404), error response body matches error-response-schema.json.

5. Configuration Management

All configuration is externalized to properties files — there is zero hard-coding in the framework. Both the UI and API modules read their configuration via the ConfigReader class in the Common module.

5.1 UI Configuration

File: UIAutomation/src/test/resources/config.properties

Property	Example Value	Purpose
browser	chrome	Browser to use for test execution
app.url	https://automationexercise.com	Base URL of the application under test
implicit.wait	10	Implicit wait timeout in seconds
test.user.email	testuser@example.com	Valid login email for TC2
test.user.password	password123	Valid login password for TC2
extent.report.path	reports/ExtentReport.html	Output path for Extent HTML report

5.2 API Configuration

File: APIAutomation/src/test/resources/config.properties

Property	Example Value	Purpose
api.base.url	https://automationexercise.com	Base URL for all API requests
test.user.name	John Doe	Full name for user creation
test.user.email	john.doe@test.com	Email for user creation/deletion
test.user.password	Test123	Password for user operations
test.search.term.top	top	Search term for TC2 scenario 1
test.search.term.invalid	nonexistentproduct123	Invalid term for TC6 negative test

5.3 ConfigReader Class

ConfigReader loads properties at class initialization using a static block, ensuring the file is read only once per JVM lifecycle. If any requested property key is not found, it throws a RuntimeException immediately — this design ensures misconfiguration is detected at startup, not mid-test.

```
// Usage in any class:
String browser  = ConfigReader.get("browser");
String baseUrl  = ConfigReader.get("api.base.url");
```

6. Parallel Execution Strategy

The framework is designed to support parallel test execution at the scenario level. The UI module's TestRunner overrides the Cucumber DataProvider to enable parallel execution, while the Common module's ThreadLocal WebDriver pattern guarantees thread isolation.

Configuration Item	Value / Setting
Parallel DataProvider	@DataProvider(parallel = true) in TestRunner
Thread Isolation Mechanism	ThreadLocal<WebDriver> in DriverManager
Thread-per-scenario	Yes — each Cucumber scenario gets its own thread and WebDriver instance
API Parallel Execution	TestNG parallel="tests" thread-count="3" in testng-api.xml
Maven Surefire Parallel	parallel=methods, threadCount=4 in parent pom.xml

6.1 ThreadLocal WebDriver Lifecycle

- **DriverFactory.initDriver() creates a new WebDriver for the current thread. ConfigReader supplies the browser name from config.properties.**@Before Hook:
- **All Page Object classes call DriverManager.getDriver() to retrieve the thread-local instance. No driver reference is ever shared between scenarios.**During Execution:
- **DriverManager.getDriver().quit() closes the browser; DriverManager.unload() removes the ThreadLocal reference, preventing memory leaks between scenarios.**@After Hook:

6.2 Execution Commands

Run all tests:

```
mvn clean test
```

Run UI tests with parallel execution:

```
cd UIAutomation && mvn clean test -Dtestng.xml=testng-ui.xml
```

Run API tests only:

```
cd APIAutomation && mvn clean test -Dtestng.xml=testng-api.xml
```

Run specific tag:

```
mvn test -Dcucumber.options="--tags @TC1"
```

Run with custom thread count:

```
mvn test -DthreadCount=8
```

7. Reporting and Logging

The framework provides a multi-layered reporting and logging strategy. Every test run produces Cucumber HTML and JSON reports, Allure interactive reports (API), Extent HTML reports (UI), and structured Log4j2 log files. Screenshots are automatically embedded in reports on UI test failure.

7.1 Extent Reports (UI Module)

Extent Reports are integrated into the UI module via ExtentManager and ExtentTestManager. The @Before hook creates a test node for each scenario; the @After hook marks it passed or failed with an embedded Base64 screenshot. The report is flushed to disk after each scenario.

- Location: reports/ExtentReport.html
- Format: Interactive HTML with dashboard, timeline, and embedded screenshots
- Thread-safe via ThreadLocal<ExtentTest> in ExtentTestManager

7.2 Allure Reporting (API Module)

Allure is integrated into the API module via the allure-testng adapter. The adapter automatically collects test data and writes result files to target/allure-results/ after each run.

- Per-scenario step-by-step breakdown with pass/fail/skip status
- Execution timeline showing scenario durations
- Tag-based filtering and suite grouping

```
# Generate and serve Allure report:
allure serve target/allure-results
```

7.3 Cucumber Reports (Both Modules)

Plugin	Output Location	Purpose
pretty	Console / stdout	Formatted step-by-step output during the run
html	target/cucumber-reports/cucumber.html	Human-readable pass/fail report with step details
json	target/cucumber-reports/cucumber.json	Machine-readable report for CI pipeline integration

7.4 Screenshot Capture on Failure

- Selenium's TakesScreenshot interface captures the screen as a byte array
- Screenshot saved to Screenshots/<scenarioName>_<timestamp>.png
- Byte array attached to the Cucumber scenario via scenario.attach() — embedded in the HTML report
- Base64-encoded image passed to ExtentTestManager for embedding in the Extent Report
- Every failure is self-contained — investigation does not require access to the test machine

7.5 Log4j2 Logging

Log4j2 is configured via log4j2.xml. The Common module's LogManager class provides Logger instances to any class in both modules using a single centralized logging configuration. Logging is applied across driver initialization, each step execution, page object actions, API requests and responses, and all error conditions.

- Log output directed to target/logs/automation.log
- Console appender also active during execution for real-time monitoring

7.6 Report Summary

Artifact	Location	Module
Extent HTML Report	reports/ExtentReport.html	UI
Allure Results	target/allure-results/	API
Cucumber HTML	target/cucumber-reports/cucumber.html	UI & API
Cucumber JSON	target/cucumber-reports/cucumber.json	UI & API
TestNG Report	target/surefire-reports/index.html	UI & API
Log File	target/logs/automation.log	UI & API
Screenshots	Screenshots/<name>_<timestamp>.png	UI

8. Technology Stack

Technology / Library	Version	Module(s)	Role / Purpose
Java (JDK)	8+	All	Primary programming language for all framework source code.
Maven	3.6+	All	Build tool, dependency management, and multi-module project orchestration.
Selenium WebDriver	4.18.1	UI	Browser automation library for all UI test interactions.
WebDriverManager	5.8.0	UI	Automatic browser driver binary resolution and management.
Cucumber (Java + TestNG)	7.15.0	UI, API	BDD framework mapping Gherkin feature file scenarios to Java step definitions.
TestNG	7.9.0	UI, API	Test execution engine providing parallel DataProvider, assertions, and suite management.
RestAssured	5.4.0	API	HTTP client for constructing and executing REST API test requests and validations.
RestAssured JSON Schema Validator	5.4.0	API	JSON schema validation against stored schema files for contract testing.
Log4j2 + SLF4J	2.22.1	All (Common)	Structured application logging to console and file via LogManager.
Allure TestNG	2.24.0	API	Rich interactive test reporting with history, attachments, and categories.
Extent Reports	5.1.1	UI	HTML reporting with dashboard, timeline, and embedded screenshots.
Apache POI	5.2.5	Common	Excel file read/write operations for data-driven test support.
Apache Commons Lang3	3.14.0	Common	Utility functions for string manipulation, random data, etc.
Git	—	All	Version control. Repository: github.com/sailubandi/Precision-Unified-Automation-Framework
Eclipse / IntelliJ IDEA	—	—	IDE with Cucumber and Gherkin plugin support for feature file editing.

9. CI/CD Integration & Best Practices

9.1 GitHub Actions

```
name: Test Suite
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Set up JDK 8
        uses: actions/setup-java@v2
        with: { java-version: "8" }
      - name: Run tests
        run: mvn clean test -Dmaven.test.failure.ignore=true
      - name: Generate Allure Report
        run: allure generate target/allure-results --clean -o allure-report
```

9.2 Jenkins Pipeline

```
pipeline {
  agent any
  stages {
    stage("Test") {
      steps { sh "mvn clean test -Dmaven.test.failure.ignore=true" }
      post { always { publishHTML([reportDir: "target/cucumber-reports",
             reportFiles: "cucumber.html"]) } }
    }
  }
}
```

9.3 Best Practices Applied

- **All UI locators and page interactions are encapsulated in dedicated Page Object classes, isolating test logic from element selectors.**Page Object Model:
- **UserDataFactory generates complete, consistent test data objects — avoiding repeated inline data construction in step definitions.**Factory Pattern:
- **DriverManager stores WebDriver in ThreadLocal to enable safe parallel execution without shared mutable state.**ThreadLocal Pattern:
- **RequestBuilder uses the Builder pattern for clean, readable API request construction.**Builder Pattern:
- **All environment-specific values (URLs, credentials, browser type, timeouts) are read from config.properties files.**Zero Hard-Coding:
- **UI, API, and Common utilities are in separate Maven modules — a change in one module does not affect the others.**Separation of Concerns:
- **Test cases follow a TC1, TC2... convention; class names directly reflect their scenario (TC1_RegisterSteps.java).**Consistent Naming:
- **Ensures every UI test failure is self-documented without manual intervention.**Automatic Screenshot on Failure:

10. Team Information

Team Member	Email	Phone
Bandi Poorna Sri Sailaja	sailubandi33@gmail.com	6300655517
Andra Tulasi Lakshmi Tanya	tanyakonapala@gmail.com	6302601497
Nerella N S Venkata Siva Sai Rihitha	rihitha9@gmail.com	9290069793
Rambha Rasmitha Lekha	rambhasrasmi7024@gmail.com	91214 87799

Team Name	QAlasers
GitHub Repository	https://github.com/sailubandi/Precision-Unified-Automation-Framework
Application Under Test	https://automationexercise.com
Submitted To	Veeva Systems
Version	1.0.0
Last Updated	April 2026